

Interview-Style Programming Questions: Basic Math and Logic

1. Area of Square

Question: Calculate the area of a square. - **Formula:** $\text{Area} = \text{side} \times \text{side}$ - **Input:** - Side = 5 - **Output:** - Area of square is: 25

2. Area of Rectangle

Question: Calculate the area of a rectangle. - **Formula:** $\text{Area} = \text{length} \times \text{breadth}$ - **Input:** - Length = 6 - Breadth = 4 - **Output:** - Area of rectangle is: 24

3. Area of Triangle

Question: Calculate the area of a triangle using base and height. - **Formula:** $\text{Area} = (1/2) \times \text{base} \times \text{height}$ - **Input:** - Base = 8 - Height = 5 - **Output:** - Area of triangle is: 20.0

4. Perimeter of Square

Question: Calculate the perimeter of a square. - **Formula:** $\text{Perimeter} = 4 \times \text{side}$ - **Input:** - Side = 6 - **Output:** - Perimeter of square is: 24

5. Perimeter of Rectangle

Question: Calculate the perimeter of a rectangle. - **Formula:** $\text{Perimeter} = 2 \times (\text{length} + \text{breadth})$ - **Input:** - Length = 5 - Breadth = 3 - **Output:** - Perimeter of rectangle is: 16

6. Perimeter of Triangle

Question: Calculate the perimeter of a triangle. - **Formula:** $\text{Perimeter} = \text{side1} + \text{side2} + \text{side3}$ - **Input:** - Side1 = 5, Side2 = 6, Side3 = 7 - **Output:** - Perimeter of triangle is: 18

7. Break Amount into 1000s, 500s, and Remaining Change

Question: Break the total amount into denominations. - **Input:** - Amount = 3700 - **Output:** - 1000s: 3 - 500s: 1 - Remaining: 200

8. Convert Seconds into Hours, Minutes, and Seconds

Question: Convert total seconds into hours, minutes, and seconds. - **Input:** - Total seconds = 3672 - **Output:** - Hours: 1 - Minutes: 1 - Seconds: 12

9. Sum of Marks (Maths, Physics, Chemistry)

Question: Calculate the sum of marks in 3 subjects. - **Input:** - Maths = 85 - Physics = 90 - Chemistry = 88 - **Output:** - Total marks: 263

10. Average of Marks (Maths, Physics, Chemistry)

Question: Calculate the average of marks in 3 subjects. - **Input:** - Maths = 85 - Physics = 90 - Chemistry = 88 - **Output:** - Average marks: 87.67

Interview-Style Questions Based on Conditional Statements

1. Check Even or Odd

Question: Determine whether a number is even or odd. **Explanation:** A number is even if it is divisible by 2. Otherwise, it's odd. - **Input:** Number = 6 - **Output:** Even number

2. Divisible by 5 but Not by 10

Question: Check if a number is divisible by 5 but not by 10. **Explanation:** Use modulo (%) to check if the number % 5 == 0 and number % 10 != 0. - **Input:** Number = 25 - **Output:** Satisfy

3. Biggest Among Two Numbers

Question: Find the biggest number among two. **Explanation:** Use comparison operators (>) to check which number is greater. - **Input:** A = 4, B = 7 - **Output:** Biggest is: 7

4. Smallest Among Two Numbers

Question: Find the smallest number among two. **Explanation:** Use comparison operators (<) to find the smaller value. - **Input:** A = 4, B = 7 - **Output:** Smallest is: 4

5. Divisible by 2, 3, and 6

Question: Check if a number is divisible by 2, 3, and 6. **Explanation:** If a number is divisible by both 2 and 3, it is also divisible by 6. - **Input:** Number = 18 - **Output:** Satisfy

6. Voting Eligibility

Question: Check if a person is eligible to vote (age >= 18). **Explanation:** A person is eligible to vote if their age is 18 or above. - **Input:** Age = 19 - **Output:** Eligible to vote

7. Student Pass/Fail Based on All Subjects ≥ 35

Question: Check if a student passed all subjects (maths, physics, chemistry).

Explanation: Student passes only if marks in all subjects are 35 or more. - **Input:** Maths = 40, Physics = 36, Chemistry = 30 - **Output:** Fail

8. Student Pass if Passed Any One Subject (≥ 35)

Question: Check if the student passed at least one subject. **Explanation:** Use logical OR to check if any one subject has marks ≥ 35 . - **Input:** Maths = 20, Physics = 38, Chemistry = 25 - **Output:** Pass

9. Student Pass if Passed Any Two Subjects

Question: Check if the student passed any two out of three subjects. **Explanation:** Use a counter or logical conditions to verify two subjects ≥ 35 . - **Input:** Maths = 40, Physics = 20, Chemistry = 36 - **Output:** Pass

10. Biggest Among Three Numbers

Question: Find the biggest number among three. **Explanation:** Compare each pair of numbers using if-else conditions. - **Input:** A = 7, B = 4, C = 9 - **Output:** Biggest is: 9

11. Smallest Among Three Numbers

Question: Find the smallest number among three. **Explanation:** Use comparison logic to determine the minimum value. - **Input:** A = 7, B = 4, C = 9 - **Output:** Smallest is: 4

12. Perfect Square or Not

Question: Check if a number is a perfect square. **Explanation:** A number is a perfect square if the square of its square root equals the number. - **Input:** Number = 49 - **Output:** Perfect square

13. Cars Required for Members (Max 5 per car)

Question: Calculate how many cars are needed for a given number of people. **Explanation:** Divide total people by 5 and round up using ceiling logic. - **Input:** Members = 17 - **Output:** Cars needed = 4

14. Second Biggest Among Three Numbers

Question: Find the second largest number among three inputs. **Explanation:** Use sorting or nested conditions to find the second largest value. - **Input:** A = 10, B = 25, C = 18 - **Output:** Second biggest: 18

15. Leap Year or Not

Question: Check if a given year is a leap year. **Explanation:** A year is a leap year if it is divisible by 4, and (not divisible by 100 unless divisible by 400). - **Input:** Year = 2024 - **Output:** Leap year

Interview-Style Programming Questions: Loops, Strings, and Number Operations

1. Print Numbers from 1 to n

Question: Write a program to print numbers from 1 to n. **Explanation:** Use a loop starting from 1 to n and print each number. - **Input:** n = 5 - **Output:** 1 2 3 4 5

2. Print Numbers from m to n

Question: Write a program to print numbers from m to n. **Explanation:** Loop from m to n and print values. - **Input:** m = 3, n = 7 - **Output:** 3 4 5 6 7

3. Print Numbers from n to 1 in Reverse

Question: Write a program to print numbers in reverse from n to 1. **Explanation:** Use a loop starting from n and decrement to 1. - **Input:** n = 5 - **Output:** 5 4 3 2 1

4. Print Numbers from n to m in Reverse

Question: Write a program to print numbers from n to m in reverse. **Explanation:** Start from n and go down to m. - **Input:** n = 10, m = 6 - **Output:** 10 9 8 7 6

5. Sum of n Natural Numbers

Question: Write a program to calculate the sum of first n natural numbers. **Explanation:** Use formula or loop to sum from 1 to n. - **Input:** n = 5 - **Output:** 15

6. Factorial of a Number

Question: Write a program to find the factorial of a number. **Explanation:** Multiply all numbers from 1 to n. - **Input:** n = 5 - **Output:** 120

7. Sum of m to n Numbers

Question: Write a program to find the sum of all numbers from m to n. **Explanation:** Loop from m to n and add values. - **Input:** m = 3, n = 6 - **Output:** 18

8. Product of m to n Numbers

Question: Write a program to find the product of numbers from m to n. **Explanation:** Loop from m to n and multiply values. - **Input:** m = 2, n = 4 - **Output:** 24

9. Print Factors of a Number

Question: Write a program to print all factors of a given number. **Explanation:** Check divisibility of number from 1 to n. - **Input:** n = 6 - **Output:** 1 2 3 6

10. Count of Factors

Question: Write a program to count how many factors a number has. **Explanation:** Increment count when divisible. - **Input:** n = 6 - **Output:** 4

11. Prime Number Check

Question: Check if a number is prime. **Explanation:** A number is prime if it has exactly 2 factors. - **Input:** n = 7 - **Output:** Prime

12. Even Numbers from m to n

Question: Print all even numbers between m and n. **Explanation:** Use loop and check if divisible by 2. - **Input:** m = 3, n = 10 - **Output:** 4 6 8 10

13. Odd Numbers from m to n

Question: Print all odd numbers between m and n. **Explanation:** Check if number % 2 != 0. - **Input:** m = 3, n = 10 - **Output:** 3 5 7 9

14. Count of Even and Odd Numbers

Question: Count how many even and odd numbers are in the range m to n. **Explanation:** Use counters for even and odd. - **Input:** m = 3, n = 7 - **Output:** Even = 2, Odd = 3

15. Reverse a String

Question: Reverse a given string. **Explanation:** Use slicing or loop. - **Input:** "hello" - **Output:** "olleh"

16. Check for Palindrome String

Question: Check if a string is a palindrome. **Explanation:** Compare string with its reverse. - **Input:** "madam" - **Output:** Palindrome

17. Sum of Digits

Question: Calculate the sum of digits of a number. **Explanation:** Use loop and % 10 to extract digits. - **Input:** 123 - **Output:** 6

18. Product of Digits

Question: Calculate the product of digits. **Explanation:** Multiply digits extracted from number. - **Input:** 123 - **Output:** 6

19. Armstrong Number Check

Question: Check if a number is an Armstrong number. **Explanation:** Sum of cube of digits equals the number. - **Input:** 153 - **Output:** Armstrong number

20. Reverse a Number

Question: Reverse the digits of a number. **Explanation:** Use loop with % and // to reverse. - **Input:** 123 - **Output:** 321

21. Palindrome Number Check

Question: Check if a number is a palindrome. **Explanation:** Compare number with its reverse. - **Input:** 121 - **Output:** Palindrome

22. Count Vowels in String

Question: Count number of vowels in a string. **Explanation:** Loop and check for a, e, i, o, u. - **Input:** "apple" - **Output:** 2

23. Count Consonants in String

Question: Count consonants in a string. **Explanation:** Check for alphabetic characters not vowels. - **Input:** "apple" - **Output:** 3

24. Count Vowels and Consonants

Question: Count vowels and consonants in input string. **Explanation:** Maintain two counters. - **Input:** "apple" - **Output:** Vowels = 2, Consonants = 3

25. Perfect Number Check

Question: Check if a number is perfect. **Explanation:** Sum of proper divisors equals the number. - **Input:** 28 - **Output:** Perfect number

26. Neon Number Check

Question: Check if a number is a neon number. **Explanation:** Square the number, sum digits, match original. - **Input:** 9 - **Output:** Neon number

27. Strong Number Check

Question: Check if a number is a strong number. **Explanation:** Sum of factorial of digits equals the number. - **Input:** 145 - **Output:** Strong number

28. Harshad Number Check

Question: Check if a number is divisible by the sum of its digits. **Explanation:** Calculate digit sum and check divisibility. - **Input:** 18 - **Output:** Harshad number

29. Fibonacci Series

Question: Print the Fibonacci series up to n terms. **Explanation:** Start with 0, 1 and continue with sum of last two. - **Input:** n = 5 - **Output:** 0 1 1 2 3

30. Check for Neon Number (Repeated)

Question: Again, check for a neon number (example). **Explanation:** Square number and sum digits. - **Input:** 9 - **Output:** Neon number

Pattern-Based Programming Questions (All 35 Questions - Interview Style)

□ Square, Rectangle, and Triangle Patterns (1–15)

1. Solid Square Pattern

Problem: Print a solid square of stars of size n.

Input: n = 4

Output:

```
* * * *
* * * *
* * * *
* * * *
```

2. Solid Rectangle Pattern

Problem: Print a solid rectangle of m rows and n columns.

Input: m = 3, n = 5

Output:

```
* * * * *
* * * * *
* * * * *
```

1. Right-Angled Triangle (Left-Aligned)

Problem: Print a left-aligned right-angled triangle.

Input: n = 5

Output:

```
*
* *
* * *
* * * *
* * * * *
```

1. Right-Angled Triangle (Right-Aligned)

Input: n = 5

Output:

```
    *
   * *
  * * *
 * * * *
* * * * *
```

1. Inverted Triangle (Left-Aligned)

Input: n = 5

Output:

```
* * * * *
* * * *
* * *
* *
*
```

1. **Inverted Triangle (Right-Aligned)**

Input: n = 5

Output:

```
* * * * *
 * * * *
  * * *
   * *
    *
```

1. **Centered Pyramid Pattern**

Input: n = 4

Output:

```
    *
   * * *
  * * * * *
 * * * * * * *
```

1. **Diamond Pattern**

Input: n = 3

Output:

```
    *
   * * *
  * * * * *
 * * * *
  * * *
   * *
```

1. **Butterfly Pattern**

Input: n = 4

Output:

```
*       *
* *   * *
* * * * *
* *   * *
*       *
```

1. **Left-Aligned Half Diamond**

Input: n = 4

Output:

```
*
* *
* * *
* * * *
* * *
* *
*
```

1. **Right-Aligned Half Diamond**

Input: n = 4

Output:

```

      *
     * *
    * * *
   * * * *
  * * * *
 * * * *
* * * *

```

1. Sandglass Pattern

Input: $n = 4$

Output:

```

* * * *
 * * *
  * *
   *
  * *
 * * *
* * * *

```

1. Increasing Width Triangle

Input: $n = 5$

Output:

```

*
* *
* * *
* * * *
* * * * *

```

1. Decreasing Width Triangle

Input: $n = 5$

Output:

```

* * * * *
* * * *
* * *
* *
*

```

1. Right-Aligned Hill Pattern

Input: $n = 4$

Output:

```

      *
     * *
    * * *
   * * * *

```

□ Hollow Patterns (16–25)

1. Hollow Square Pattern

Problem: Print a hollow square of stars of size n .

Input: n = 4

Output:

```
* * * *
*      *
*      *
* * * *
```

1. **Hollow Rectangle Pattern**

Problem: Print a hollow rectangle of m rows and n columns.

Input: m = 4, n = 5

Output:

```
* * * * *
*      *
*      *
* * * * *
```

1. **Hollow Right-Angled Triangle (Left-Aligned)**

Input: n = 5

Output:

```
*
* *
*  *
*    *
* * * * *
```

1. **Hollow Right-Angled Triangle (Right-Aligned)**

Input: n = 5

Output:

```
      *
     * *
    *  *
   *   *
  *    *
 * * * * *
```

1. **Hollow Inverted Triangle (Left-Aligned)**

Input: n = 5

Output:

```
* * * * *
*      *
*    *
*  *
* *
*
```

1. **Hollow Inverted Triangle (Right-Aligned)**

Input: n = 5

Output:

```
* * * * *
      *
     * *
```

```

    * *
      *

```

1. **Hollow Pyramid Pattern**

Input: n = 4

Output:

```

      *
     * *
    *   *
   *     *
  * * * * *

```

1. **Hollow Diamond Pattern**

Input: n = 3

Output:

```

    *
   * *
  *   *
 *     *
  *   *
   * *
    *

```

1. **Hollow Butterfly Pattern**

Input: n = 4

Output:

```

*       *
* *   * *
*  *  *
*       *
*  *  *
* *   * *
*       *

```

1. **Hollow Hourglass Pattern**

Input: n = 5

Output:

```

* * * * *
*       *
  *     *
   *
  *     *
*       *
* * * * *

```

□ **Number-Based Patterns (26–35)**

1. **Increasing Number Triangle**

Problem: Print numbers from 1 to n in triangle form.

Input: n = 5

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

1. **Repeating Row Number Triangle**

Input: n = 5

Output:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

1. **Continuous Number Triangle**

Input: n = 4

Output:

```
1
2 3
4 5 6
7 8 9 10
```

1. **Reverse Row Number Triangle**

Input: n = 5

Output:

```
1
2 1
3 2 1
4 3 2 1
5 4 3 2 1
```

1. **Inverted Number Triangle**

Input: n = 5

Output:

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

1. **Right-Aligned Number Triangle**

Input: n = 5

Output:

```
      1
     1 2
    1 2 3
   1 2 3 4
  1 2 3 4 5
```


1. **Pyramid Number Pattern**

Input: n = 4

Output:

```
    1
  1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
```

1. **Even Number Triangle**

Input: n = 5

Output:

```
2
2 4
2 4 6
2 4 6 8
2 4 6 8 10
```

1. **Odd Number Triangle**

Input: n = 5

Output:

```
1
1 3
1 3 5
1 3 5 7
1 3 5 7 9
```

1. **Pascal's Triangle**

Input: n = 5

Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Interview Questions – Logic-Based Programs

□ Prime and Armstrong Logic Questions

1. Print All Prime Numbers from m to n

Problem: Given a range from m to n, print all prime numbers in that range.

Input: m = 10, n = 30

Output: 11 13 17 19 23 29

Explanation: A prime number has only two factors: 1 and itself.

2. Count of All Prime Numbers from m to n

Problem: Count how many prime numbers are there between m and n.

Input: m = 1, n = 10

Output: 4

Explanation: Prime numbers are: 2, 3, 5, 7

3. Print All Armstrong Numbers in a Range

Problem: Print all Armstrong numbers between m and n.

Input: m = 1, n = 500

Output: 1 153 370 371 407

Explanation: Armstrong number = sum of each digit raised to the power of number of digits.

4. First Prime Number from m to n

Problem: Find the first prime number in the given range.

Input: m = 10, n = 25

Output: 11

5. Last Prime Number from m to n

Problem: Find the last prime number in the given range.

Input: m = 10, n = 25

Output: 23

6. First Vowel in a Name

Problem: Given a string, find the first vowel in the string.

Input: name = "Krishna"

Output: i

Explanation: First vowel from left is 'i'

7. Last Vowel in a Name

Problem: Given a string, find the last vowel in the string.

Input: name = "Ramakrishna"

Output: a

Explanation: Last vowel is 'a'

8. Print All Even Numbers Using Continue

Problem: Use continue statement to skip odd numbers and print only even numbers between 1 and n.

Input: n = 10

Output: 2 4 6 8 10

9. Print All Odd Numbers Using Continue

Problem: Use continue statement to skip even numbers and print only odd numbers.

Input: n = 10

Output: 1 3 5 7 9

10. Count of Prime and Composite Numbers from m to n

Problem: Count how many are prime and how many are composite numbers in range m to n.

Input: m = 1, n = 10

Output: Prime: 4, Composite: 4

Explanation: Prime: 2,3,5,7 | Composite: 4,6,8,9

Function-Based Interview Questions – Logic & Practice

1. Add Two Numbers Using Function

Problem: Write a function that takes two numbers as arguments and returns their sum.

Explanation: The function should accept two inputs and return the result of their addition.

Input:

Enter first number: 10
Enter second number: 20

Output:

Sum = 30

2. Check Even or Odd Using Function

Problem: Write a function that determines whether a number is even or odd.

Explanation: Even numbers are divisible by 2. The function should return “Even” or “Odd”.

Input:

Enter number: 15

Output:

15 is Odd

3. Check Leap Year Using Function

Problem: Write a function that checks if a given year is a leap year.

Explanation: A leap year is divisible by 4 but not by 100 unless also divisible by 400.

Input:

Enter year: 2020

Output:

2020 is a Leap Year

4. Check Prime Number Using Function

Problem: Write a function that checks whether a given number is prime.

Explanation: A prime number has only two factors: 1 and itself.

Input:

Enter number: 13

Output:

13 is a Prime Number

5. Print Armstrong Numbers from m to n Using Function

Problem: Write a function that prints all Armstrong numbers in a given range from m to n.

Explanation: Armstrong number = sum of digits raised to the power of number of digits.

Input:

m = 100

n = 500

Output:

Armstrong Numbers between 100 and 500: 153 370 371 407

☐ Recursion-Based Problems

6. Factorial Using Recursion

Problem: Write a recursive function to calculate the factorial of a number.

Explanation: $\text{factorial}(n) = n * \text{factorial}(n - 1)$, with base case $\text{factorial}(0) = 1$.

Input:

Enter number: 5

Output:

Factorial = 120

7. Fibonacci Series Using Recursion

Problem: Print the Fibonacci series up to n terms using recursion.

Explanation: $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$ with base cases $\text{fibonacci}(0)=0$, $\text{fibonacci}(1)=1$.

Input:

Enter terms: 5

Output:

Fibonacci Series: 0 1 1 2 3

8. Sum of Digits Using Recursion

Problem: Write a recursive function to find the sum of digits of a number.

Explanation: $\text{sum}(n) = n \% 10 + \text{sum}(n // 10)$

Input:

Enter number: 123

Output:

Sum of digits = 6

9. Reverse a Number Using Recursion

Problem: Write a recursive function to reverse the digits of a number.

Explanation: Keep multiplying result by 10 and adding the current digit.

Input:

Enter number: 1234

Output:

Reversed Number = 4321

10. Check Palindrome Using Recursion

Problem: Write a recursive function to check if a string is a palindrome.

Explanation: Compare first and last characters and recurse on the substring.

Input:

Enter string: madam

Output:

The string is a palindrome.

String and Text Manipulation – Interview Questions

1. Remove Spaces from Given Text

Problem: Write a function to remove all spaces from the input string. **Explanation:** Remove any whitespace characters. **Input:** "he llo wor ld" **Output:** "helloworld"

2. Reverse a String

Problem: Write a function to reverse the characters in a string. **Input:** "hello" **Output:** "olleh"

3. Reverse a String After Removing Spaces

Problem: Write a function to reverse a string after removing all spaces. **Input:** "he llo world" **Output:** "dlrowolleh"

4. Convert Snake Case to Camel Case

Problem: Convert a string from snake_case to camelCase. **Input:** "my_variable_name" **Output:** "myVariableName"

5. Convert Snake Case to Pascal Case

Problem: Convert a string from snake_case to PascalCase. **Input:** "my_variable_name" **Output:** "MyVariableName"

6. Convert Camel Case to Snake Case

Problem: Convert a string from camelCase to snake_case. **Input:** "myVariableName" **Output:** "my_variable_name"

7. Convert Camel Case to Pascal Case

Problem: Convert a string from camelCase to PascalCase. **Input:** "myVariable" **Output:** "MyVariable"

8. Convert Pascal Case to Camel Case

Problem: Convert a string from PascalCase to camelCase. **Input:** "MyVariable"

Output: "myVariable"

9. Convert Pascal Case to Snake Case

Problem: Convert a string from PascalCase to snake_case. **Input:** "MyVariable"

Output: "my_variable"

10. Convert Text to Camel Case

Problem: Convert a space-separated sentence into camelCase. **Input:** "hello world example" **Output:** "helloWorldExample"

11. Convert Text to Snake Case

Problem: Convert a space-separated sentence into snake_case. **Input:** "hello world example" **Output:** "hello_world_example"

12. Convert Text to Pascal Case

Problem: Convert a space-separated sentence into PascalCase. **Input:** "hello world example" **Output:** "HelloWorldExample"

13. Swap Upper and Lower Case

Problem: Swap the case of each letter in a given string. **Input:** "HeLLo" **Output:** "hE110"

14. Separate Digits from Text

Problem: Extract all digits from a given alphanumeric string. **Input:** "abc123d4" **Output:** "1234"

15. Print Uppercase, Lowercase, Digits, and Special Characters Separately

Problem: Print each type of character separately from the string. **Input:** "Abc123!@#" **Output:**

Uppercase: A
Lowercase: b c
Digits: 1 2 3
Special Characters: ! @ #

16. Count of Uppercase, Lowercase, Digits, and Special Characters

Problem: Count each type of character in a string. **Input:** "AbC@123x!" **Output:**

Uppercase: 2
Lowercase: 1
Digits: 3
Special Characters: 2

17. Check Password Strength

Problem: Check if a password contains at least one uppercase, one lowercase, one digit, and one special character. **Input:** "Pass123!" **Output:** "Strong Password"

18. Remove Duplicates in a Given Input

Problem: Remove duplicate characters from a string. **Input:** "aabbcc" **Output:** "abc"

19. Print Duplicates in a Given String

Problem: Identify and print duplicate characters in a string. **Input:** "aabbccde" **Output:**
a b c

20. Print Next Characters in a Given String

Problem: Replace each character in the string with its next character. **Input:** "abc" **Output:** "bcd"

List-Based Interview Questions – Logic & Practice

1. Add an Element to a List

Problem: Write a function to add an element to a list. **Explanation:** Use `append()` to add the element to the end. **Input:** [1, 2, 3], add 4 **Output:** [1, 2, 3, 4]

2. Remove an Element from a List

Problem: Write a function to remove a specific element from a list. **Explanation:** Use `remove()` or `pop()` if index is known. **Input:** [1, 2, 3, 4], remove 3 **Output:** [1, 2, 4]

3. Find Maximum in List

Problem: Find the maximum value in a list. **Explanation:** Use `max()` or iterate manually. **Input:** [4, 7, 1, 9] **Output:** 9

4. Find Minimum in List

Problem: Find the minimum value in a list. **Explanation:** Use `min()` or iterate manually. **Input:** [4, 7, 1, 9] **Output:** 1

5. Sum of All Elements in List

Problem: Write a function to find the sum of all list elements. **Explanation:** Use `sum()` or loop to add all items. **Input:** [1, 2, 3] **Output:** 6

6. Count Occurrence of an Element

Problem: Count how many times a value appears in a list. **Explanation:** Use `count()` method. **Input:** [1, 2, 2, 3, 2], value 2 **Output:** 3

7. Reverse a List

Problem: Write a function to reverse the order of list elements. **Explanation:** Use slicing or `reverse()` method. **Input:** [1, 2, 3] **Output:** [3, 2, 1]

8. Sort a List

Problem: Write a function to sort a list in ascending order. **Explanation:** Use `sort()` or `sorted()`. **Input:** [4, 1, 3, 2] **Output:** [1, 2, 3, 4]

9. Remove Duplicates from a List

Problem: Eliminate duplicate values. **Explanation:** Use `set()` or manual loop. **Input:** [1, 2, 2, 3] **Output:** [1, 2, 3]

10. Merge Two Lists

Problem: Merge two lists into one. **Explanation:** Use `+` operator or `extend()`. **Input:** [1, 2], [3, 4] **Output:** [1, 2, 3, 4]

11. Find Common Elements in Two Lists

Problem: Return common elements between two lists. **Explanation:** Use `set()` and `&` or loops. **Input:** [1, 2, 3], [2, 3, 4] **Output:** [2, 3]

12. Print Even Numbers in a List

Problem: Print only even numbers from a list. **Explanation:** Use modulo condition in a loop. **Input:** [1, 2, 3, 4] **Output:** [2, 4]

13. Print Odd Numbers in a List

Problem: Print only odd numbers from a list. **Input:** [1, 2, 3, 4] **Output:** [1, 3]

14. Check if List is Palindrome

Problem: Check if the list reads the same forwards and backwards. **Input:** [1, 2, 1] **Output:** True

15. Count Positive, Negative, Zero

Problem: Count the number of positive, negative and zero values in a list. **Input:** [0, -1, 2, -3, 4] **Output:** Positive: 2, Negative: 2, Zero: 1

16. Find Second Largest Number in List

Problem: Find the second highest value. **Input:** [1, 3, 4, 5, 0] **Output:** 4

17. Find Second Smallest Number in List

Problem: Find the second lowest value. **Input:** [5, 1, 4, 2, 3] **Output:** 2

18. Copy List to Another List

Problem: Copy the contents of one list to another. **Explanation:** Use slicing or copy().
Input: [1, 2, 3] **Output:** [1, 2, 3]

19. Print All Prime Numbers in List

Problem: Print all prime numbers from a list. **Input:** [1, 2, 3, 4, 5] **Output:** [2, 3, 5]

20. Replace All Zeroes with a Given Number

Problem: Replace every zero with a specific value (e.g., -1). **Input:** [0, 2, 0, 4], replace with -1 **Output:** [-1, 2, -1, 4]

21. Check if All Elements Are Same

Problem: Check whether all elements in the list are identical. **Input:** [5, 5, 5, 5] **Output:** True

22. Find Frequency of All Elements

Problem: Return a dictionary with the frequency of each element. **Input:** [1, 2, 2, 3, 1] **Output:** {1: 2, 2: 2, 3: 1}

23. Flatten a Nested List

Problem: Convert a nested list into a single list. **Input:** [[1, 2], [3, 4]] **Output:** [1, 2, 3, 4]

24. Split a List into Even and Odd Lists

Problem: Separate even and odd numbers into different lists. **Input:** [1, 2, 3, 4, 5]

Output: Even: [2, 4], Odd: [1, 3, 5]

25. Find Pair of Elements with Given Sum

Problem: Find all pairs in the list whose sum equals a given value. **Input:** [1, 2, 3, 4], sum = 5 **Output:** [(1, 4), (2, 3)]

26. Remove All Odd Numbers

Problem: Remove all odd numbers from the list. **Input:** [1, 2, 3, 4, 5] **Output:** [2, 4]

27. Remove All Even Numbers

Problem: Remove all even numbers from the list. **Input:** [1, 2, 3, 4, 5] **Output:** [1, 3, 5]

28. Multiply All Elements by a Number

Problem: Multiply every element in the list by a fixed number. **Input:** [1, 2, 3], multiply by 2 **Output:** [2, 4, 6]

29. Find Difference Between Max and Min

Problem: Return the difference between the largest and smallest element. **Input:** [4, 2, 7, 1] **Output:** 6

30. Check if a List is Empty

Problem: Write a function that returns True if the list is empty, else False. **Input:** [] **Output:** True

31. Insert Element at Specific Index

Problem: Insert a value at a specific position. **Input:** [1, 2, 4], insert 3 at index 2

Output: [1, 2, 3, 4]

32. Remove All Instances of a Value

Problem: Remove all occurrences of a specific value. **Input:** [1, 2, 2, 3], remove 2

Output: [1, 3]

33. Get Index of an Element

Problem: Return the index of a given value. **Input:** [10, 20, 30], find index of 20

Output: 1

34. Square All Elements in a List

Problem: Return a list with each element squared. **Input:** [1, 2, 3] **Output:** [1, 4, 9]

35. Filter Out Negative Numbers

Problem: Remove all negative values from the list. **Input:** [-1, 2, -3, 4] **Output:** [2, 4]

36. Get Elements Greater Than a Value

Problem: Return elements greater than a specified number. **Input:** [1, 5, 8, 3], greater than 4 **Output:** [5, 8]

37. Find Duplicates in List

Problem: Return a list of duplicated values. **Input:** [1, 2, 2, 3, 3, 4] **Output:** [2, 3]

38. Rotate List Elements Right

Problem: Rotate list by k positions to the right. **Input:** [1, 2, 3, 4], k = 2 **Output:** [3, 4, 1, 2]

39. Check If List Contains a Value

Problem: Return True if list contains a specific value. **Input:** [1, 2, 3], check 2

Output: True

40. Chunk List into Smaller Lists

Problem: Break a list into chunks of given size. **Input:** [1, 2, 3, 4, 5, 6], chunk size

2 **Output:** [[1, 2], [3, 4], [5, 6]]

Set and Dictionary Interview Questions – Problem Statements and Explanations

Set-Based Questions

1. Add Element to a Set

Problem: Add an item to a given set.

Explanation: Use `add()` method.

Input: {1, 2, 3}, add 4

Output: {1, 2, 3, 4}

2. Remove Element from Set

Problem: Remove a specific element from a set.

Explanation: Use `remove()` or `discard()` to avoid `KeyError`.

Input: {1, 2, 3}, remove 2

Output: {1, 3}

3. Union of Two Sets

Problem: Find union of two sets.

Explanation: Use `|` operator or `union()` method.

Input: {1, 2}, {2, 3}

Output: {1, 2, 3}

4. Intersection of Sets

Problem: Find common elements in two sets.

Explanation: Use `&` operator or `intersection()`.

Input: {1, 2}, {2, 3}

Output: {2}

5. Difference of Sets

Problem: Elements present in first set but not in second.

Explanation: Use `-` or `difference()` method.

Input: {1, 2, 3}, {2, 3}

Output: {1}

6. Check Subset

Problem: Check if one set is a subset of another.

Explanation: Use `issubset()` method.

Input: {1, 2}, {1, 2, 3}

Output: True

7. Set Length

Problem: Find number of elements in set.

Explanation: Use `len()`.

Input: {1, 2, 3}

Output: 3

8. Clear a Set

Problem: Remove all elements from a set.

Explanation: Use `clear()`.

Input: {1, 2, 3}

Output: set()

9. Symmetric Difference

Problem: Find elements in either set but not in both.

Explanation: Use `^` or `symmetric_difference()`

Input: {1, 2, 3}, {2, 3, 4}

Output: {1, 4}

10. Convert List to Set

Problem: Remove duplicates using set.

Explanation: Use `set()` constructor.

Input: [1, 2, 2, 3]

Output: {1, 2, 3}

Dictionary-Based Questions

11. Create a Dictionary from Two Lists

Problem: Combine two lists into a dictionary.

Explanation: Use `zip()` function.

Input: ["a", "b"], [1, 2]

Output: {"a": 1, "b": 2}

12. Update Dictionary Value

Problem: Change value for a specific key.

Explanation: Use assignment `dict[key] = value`.

Input: {"a": 1}, update a to 2

Output: {"a": 2}

13. Remove Key from Dictionary

Problem: Delete a key-value pair.

Explanation: Use del or pop() method.

Input: {"a": 1, "b": 2}, remove b

Output: {"a": 1}

14. Check Key Existence

Problem: Verify if a key exists.

Explanation: Use in operator.

Input: {"x": 1}, key = "x"

Output: True

15. Iterate Over Dictionary

Problem: Print all keys and values.

Explanation: Use .items() in loop.

Input: {"a": 10, "b": 20}

Output: a 10, b 20

16. Dictionary Length

Problem: Count total key-value pairs.

Explanation: Use len() function.

Input: {"x": 1, "y": 2}

Output: 2

17. Merge Two Dictionaries

Problem: Combine two dictionaries.

Explanation: Use unpacking or update().

Input: {"a": 1}, {"b": 2}

Output: {"a": 1, "b": 2}

18. Get Value with Default

Problem: Get value or default if key not found.

Explanation: Use get() method.

Input: {"a": 1}, get "b" with default 0

Output: 0

19. Count Frequency of Elements

Problem: Count frequency using dictionary.

Input: [1, 2, 2, 3]

Output: {1: 1, 2: 2, 3: 1}

20. Invert a Dictionary

Problem: Flip keys and values.

Input: {"a": 1, "b": 2}

Output: {1: "a", 2: "b"}

21. Find Key with Maximum Value

Problem: Identify the key with the highest value.

Input: {"a": 10, "b": 20, "c": 15}

Output: "b"

22. Sort Dictionary by Values

Problem: Sort a dictionary based on its values.

Input: {"a": 3, "b": 1, "c": 2}

Output: [('b', 1), ('c', 2), ('a', 3)]

23. Create Dictionary of Squares

Problem: Create dictionary where keys are numbers and values are their squares.

Input: range(1, 4)

Output: {1: 1, 2: 4, 3: 9}

24. Filter Dictionary by Value Condition

Problem: Retain only items with value greater than a threshold.

Input: {"a": 10, "b": 5, "c": 15}, condition: > 10

Output: {"c": 15}

25. Combine Values of Duplicate Keys

Problem: Given two dictionaries, add values of common keys.

Input: {"a": 1, "b": 2}, {"a": 3, "c": 4}

Output: {"a": 4, "b": 2, "c": 4}

26. Count Word Frequency in Sentence

Problem: Count occurrences of each word in a string.

Input: "apple banana apple"

Output: {"apple": 2, "banana": 1}

27. Remove Duplicate Values from Dictionary

Problem: Remove duplicate values keeping first key only.

Input: {"a": 1, "b": 2, "c": 1}

Output: {"a": 1, "b": 2}

28. Find Common Keys in Two Dictionaries

Problem: Return keys common to both dictionaries.

Input: {"a": 1, "b": 2}, {"b": 3, "c": 4}

Output: ["b"]

29. Swap Keys and Values Safely

Problem: Flip keys and values ensuring all values are unique.

Input: {"x": 1, "y": 2}

Output: {1: "x", 2: "y"}

30. Delete Items by Value

Problem: Remove key-value pairs with specific value.

Input: {"a": 1, "b": 2, "c": 1}, value: 1

Output: {"b": 2}

Matrix Interview Questions – Problem Statements and Explanations

1. Check if a Matrix is Square

Problem: Determine if the given matrix is square.

Explanation: A matrix is square if the number of rows equals the number of columns.

Input: `[[1, 2], [3, 4]]`

Output: `True`

2. Print Diagonal Elements

Problem: Print the main diagonal elements of a square matrix.

Explanation: Elements where row index equals column index.

Input: `[[1, 2], [3, 4]]`

Output: `[1, 4]`

3. Print Anti-Diagonal Elements

Problem: Print the anti-diagonal elements.

Explanation: Elements where $\text{row} + \text{column} = n - 1$.

Input: `[[1, 2], [3, 4]]`

Output: `[2, 3]`

4. Print Non-Diagonal Elements

Problem: Print all elements except the main diagonal.

Input: `[[1, 2], [3, 4]]`

Output: `[2, 3]`

5. Print Non-Anti-Diagonal Elements

Problem: Print all elements excluding anti-diagonal.

Input: `[[1, 2], [3, 4]]`

Output: `[1, 4]`

6. Lower Triangle of Matrix

Problem: Extract elements in or below the main diagonal.

Input: `[[1, 2], [3, 4]]`

Output: `[[1, 0], [3, 4]]`

7. Upper Triangle of Matrix

Problem: Extract elements in or above the main diagonal.

Input: [[1, 2], [3, 4]]
Output: [[1, 2], [0, 4]]

8. Transpose of Matrix

Problem: Transpose the matrix (flip over the diagonal).

Input: [[1, 2], [3, 4]]
Output: [[1, 3], [2, 4]]

9. Check if Diagonal Elements are Same

Problem: Verify all diagonal elements are equal.

Input: [[5, 0], [0, 5]]
Output: True

10. Check if Anti-Diagonal Elements are Same

Problem: Verify all anti-diagonal elements are equal.

Input: [[0, 3], [3, 0]]
Output: True

11. Convert Diagonal Elements to Zero

Problem: Set all diagonal elements to zero.

Input: [[1, 2], [3, 4]]
Output: [[0, 2], [3, 0]]

12. Convert Anti-Diagonal Elements to Zero

Problem: Set all anti-diagonal elements to zero.

Input: [[1, 2], [3, 4]]
Output: [[1, 0], [0, 4]]

13. Convert Non-Diagonal Elements to Zero

Problem: Only keep diagonal elements, set rest to zero.

Input: [[1, 2], [3, 4]]
Output: [[1, 0], [0, 4]]

14. Sum of All Elements in Matrix

Problem: Calculate the total sum of matrix elements.

Input: [[1, 2], [3, 4]]
Output: 10

15. Matrix Multiplication

Problem: Multiply two matrices if their dimensions are compatible.

Input: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

Output: $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
